



Project: Agentic AI Loan Origination and Credit Assessment

INFRASTRUCTURE SUMMARY

Version: V1.0
Date: 12/08/26

Table of Contents

1	PROJECT SUMMARY ENVIRONMENT STRATEGY	1
2	DEVELOPMENT ENVIRONMENT SETUP	1
3	PROJECT MANAGEMENT SETUP	3
4	COMMUNICATION SETUP	3
5	HARDWARE AND SOFTWARE NEEDS	4
6	AZURE DEPLOYMENT TARGETS	5

1 Environment Strategy

Three environments are maintained. Each is fully isolated, separate databases, separate storage accounts, separate credentials. No environment can read another's data.

	Local	Development	Production (Demo)
Purpose	Daily coding, unit tests, agent prompt iteration	Integration testing, internal demos, automated E2E	Client sprint reviews and final showcase
Hosting	Docker Compose on each member's machine	Azure Container Apps (dev revision)	Azure Container Apps (prod revision)
Database	PostgreSQL 16 container with pgvector	Azure Database for PostgreSQL Flexible Server loan_dev	Azure Database for PostgreSQL Flexible Server loan_prod
Object storage	MinIO container (S3-compatible)	Azure Blob Storage stdev container	Azure Blob Storage stprod container
OCR	Azure Document Intelligence F0, or Tesseract offline	Azure Document Intelligence F0	Azure Document Intelligence S0
Core banking	Mock service (WireMock)	Mock service	Mock service
LLM	Real API, low-cost model	Real API, low-cost model	Real API, full-capability model
Deployed by	Developer, manually	Automatically on merge to main	Manually, on a version tag, with approval
Data	Synthetic seed dataset	Synthetic dataset, reset each sprint	Curated synthetic demo scenarios

2 Development Environment Setup

2.1 TECHNOLOGY STACK

Layer	Technology	Rationale
Web frontend	Next JS + TypeScript	Client brief specifies a web prototype; TypeScript enforces contract safety against the API layer
UI components	Tailwind CSS + shadcn/ui	Consistent design system without bespoke CSS effort
Backend API	Python + FastAPI	Native async support for long-running agent calls
Agent orchestration	[LangGraph]	Graph-based orchestration supports the supervisor-and-specialist agent topology, deterministic routing, and state persistence needed for auditability
LLM provider	[Anthropic Claude via provider abstraction layer]	Abstraction prevents vendor lock-in and allows cost and quality benchmarking between models
Document intelligence / OCR	[Azure Document Intelligence]	Structured extraction from payslips, bank statements and identity documents
Relational database	PostgreSQL	ACID guarantees required for applications, decisions and audit trails
Vector store	pgvector extension on PostgreSQL	Keeps lending-policy retrieval in the same database, removing a separate service from the local stack

Cache / task queue	Redis	Session state and background document-processing jobs
Object storage	[Azure Blob Storage / MinIO locally]	Uploaded document storage with lifecycle policies
Containerisation	Docker + Docker Compose	Guarantees environment parity across all machines

2.2 IDE AND CODE QUALITY CONFIGURATION

Item	Tool	Enforcement
Primary IDE	VS Code (shared .vscode/settings.json and recommended-extensions list)	Committed to repository
Alternative IDE	PyCharm	.editorconfig
Python lint and format	Ruff + Black	Pre-commit hook + CI
Python type checking	mypy (strict on core service modules)	CI
JavaScript / TypeScript lint and format	ESLint + Prettier	Pre-commit hook + CI
Commit hygiene	pre-commit framework; Conventional Commits	Pre-commit hook + CI
Secret leakage	gitleaks + GitHub secret scanning	Pre-commit hook + CI

2.3 CODE REPOSITORY SETUP

Item	Detail
Platform	GitHub
Repository URL	https://github.com/7-Astral/Agentic-AI-Loan-Origination-and-Credit-Assessment-Platform.git
Visibility	Public
Structure	Monorepo
Collaborators	All team members (read/write), Supervisor (read), Course Coordinator Golnoush Abaei (read), Client (read)

A monorepo was chosen over multiple repositories because the frontend, API and agent services share type definitions and change together within the same user story and cross-repository coordination overhead would exceed the isolation benefit for a team of this size.

2.4 OBSERVABILITY

Concern	Tool	Notes
Structured application logging	Python structlog, JSON output	Correlation identifier per loan application, carried into every agent node
Agent trace capture	LangSmith	Captures every prompt, tool call and node transition and free tier sufficient at project volume
Audit log	Append-only PostgreSQL table	Immutable record of every decision-affecting event
Platform metrics and errors	Azure Monitor + Application Insights	Dev and Production environments and alert rules on error rate and response time

3 Project Management Setup

Item	Detail
Tool	Jira
Board URL	https://ailoanorigination.atlassian.net/jira/software/projects/PBI/list?filter=allissues&jql=project+%3D+%22PBI%22+ORDER+BY+created+DESC&atlOrigin=eyJpIjoiOTg4MzQ3MzY2MDZINGNkZGJjMTczNjIwOWJmMmVIMjUiLCJwIjoiaj9
Members	[5] team members + supervisor (viewer) + client (viewer)
Hierarchy	Epic → Story → Sub-task, Bugs and Spikes as separate issue types
Estimation	Story points, Fibonacci scale
Sprint length	2 weeks
Prioritisation	High/Medium/Low
Definition of Done	Recorded in the Jira project description and mirrored in the pull request template
Reporting	Sprint burndown, velocity chart and sprint report exported at each sprint review

Workflow states: Backlog → Selected for Sprint → In Progress → In Review → Testing → Done

The In Review and Testing states are deliberately separated so that code review capacity is visible as its own constraint.

4 Communication Setup

4.1 CHANNELS

Channel	Tool	Used for	Participants
Team channel	Microsoft Teams	Day-to-day discussion, asynchronous stand-ups, file sharing	Team
Client meetings	Microsoft Teams (video)	Fortnightly client sessions, sprint reviews	Team + client representatives
Formal correspondence	RMIT email	Scope changes, signoffs, decisions requiring a record	Team lead, client, supervisor
Code discussion	GitHub pull request comments	Technical review, in context	Team
Ad-hoc / urgent	Teams direct message	Blockers requiring same-day response	Team

4.2 MEETING CADENCE

Meeting	Frequency	Duration	Attendees	Output
Asynchronous stand-up	Daily (weekdays)	—	Team	Written update in the general channel
Team sync	Twice weekly	45 min	Team	Action items in Jira
Supervisor meeting	Weekly	30 min	Team + supervisor	Minutes
Client meeting	Weekly	60 min	Team + client	Agenda, minutes, decisions log

Sprint planning	Per sprint	60 min	Team	Sprint backlog + planning note
Sprint review	Per sprint	60 min	Team + client	Demonstration + feedback log
Sprint retrospective	Per sprint	45 min	Team	Improvement actions

Agendas are circulated at least 24 hours before each client and supervisor meeting. Minutes are published within 24 hours afterwards, with each action item assigned to a named owner and a due date.

4.3 RESPONSE-TIME EXPECTATIONS

Agreed within the team during Sprint 0 that Teams messages are acknowledged within 24 hours on weekdays and pull request reviews are completed within 24 hours of request. Any blockers are raised immediately in the dev channel rather than held until the next sync.

5 Hardware and Software Needs

5.1 HARDWARE

Resource	Requirement	Status
Development machine (per member)	16 GB RAM minimum (the Docker stack running PostgreSQL, Redis, MinIO and two services requires approximately 8 GB of headroom), 4-core CPU, 50 GB free disk	Provided by team members
Operating system	Windows 11 (WSL2), macOS 13+, or Ubuntu 22.04+	Mixed across team, WSL2 mandated on Windows for parity
Internet	Stable connection for LLM API calls and video meetings	Personal
Backup	Repository is the source of truth; documents held in cloud storage with version history	In place

No specialised hardware (GPU) is required. all model inference is performed via hosted APIs rather than locally hosted models. This was a deliberate infrastructure decision that self-hosting an open-weight model would have introduced hardware constraints that not all team members could meet.

5.2 SOFTWARE AND SERVICES

Service	Purpose	Plan	Cost
GitHub	Source control, CI/CD	Free / Student Developer Pack	\$0
Jira	Project management	Free tier (up to 10 users)	\$0
Microsoft Teams	Communication	Free	\$0
Docker Desktop	Containerisation	Personal use	\$0
PostgreSQL, Redis, MinIO	Local data services	Open source, self-hosted	\$0
Azure Container Apps	API and agent hosting	Free monthly grant, scale-to-zero on Development	≈\$0
Azure Static Web Apps	Frontend hosting	Free tier	\$0
Azure Database for PostgreSQL	Managed database (dev + prod)	Burstable B1ms and 12-month free allowance	≈\$0
Azure Blob Storage	Document storage	Free tier (5 GB)	\$0
Azure Container	Image registry	Basic	~\$0.17/day SKU

Registry			charge
Azure Key Vault	Secrets management	Standard	\$0.03 per 10,000 operations
Microsoft Entra ID	Authentication and RBAC	Free tier	\$0
Azure Monitor / App Insights	Metrics, logs, alerts	Free ingestion allowance	\$0
Azure AI Document Intelligence	Document OCR	F0 free tier for development and = S0 for demos	\$10 per 1,000 pages
Anthropic Claude API	LLM inference	Pay-as-you-go	\$ 40 est. per sprint
Figma	Wireframes and UI design	Free tier	\$0
LangSmith	Agent tracing	Free tier	\$0

6. Azure Deployment Targets

Component	Azure service	Configuration
Frontend	Azure Static Web Apps	Deployed from the same workflow; free tier
API and agent services	Azure Container Apps	Scale-to-zero on Development to conserve credit; minimum one replica on Production
Container images	Azure Container Registry	Basic tier, images tagged with the commit SHA
Database	Azure Database for PostgreSQL Flexible Server	Burstable B1ms, pgvector enabled via server parameters, separate database per environment
Document storage	Azure Blob Storage	Separate container per environment, lifecycle policy deletes synthetic uploads after 30 days
Secrets	Azure Key Vault	Referenced by Container Apps, access via managed identity
Monitoring	Azure Monitor + Application Insights	Shared workspace, separate application per environment